



Programmiertechnik B

Praxisarbeit «Labyrinth»
Christoph Reifler



Inhaltsverzeichnis

1	Management Summary	3
1.1	Projektziel und Umfang	3
1.2	Kernfunktionen	3
1.3	Technische Umsetzung	3
1.4	Testphase	3
1.5	Projektergebnisse.....	3
1.6	Lessons Learned	4
1.7	Fazit	4
2	Aufgabenstellung und Mindestanforderungen	4
2.1	Aufgabenstellung	4
2.2	Funktionsanforderungen	5
3	Design	6
3.1	Entwurfsprozess.....	6
3.2	Gewählte Lösung.....	6
3.3	Detailbeschreibung der Lösung.....	7
4	Implementierung	9
4.1	Projektstruktur.....	9
4.2	Kernfunktionen	9
4.3	Optimierung	9
4.4	Erweiterungen	10
5	Test	12
5.1	Modultests.....	12
5.2	Systemtests.....	12
5.3	Zusammenfassung	13
6	Lessons Learned	13
7	Anhang	13

1 Management Summary

1.1 Projektziel und Umfang

Im Rahmen dieser Arbeit wurde ein textbasiertes Labyrinthspiel in der Programmiersprache C entwickelt. Zielsetzung war es, die in den Modulen Programmiertechnik A und B erworbenen Kenntnisse praxisnah einzusetzen und daraus ein interaktives Konsolenspiel zu gestalten. Der Projektumfang erstreckte sich über alle Phasen des Entwicklungsprozesses, von der Idee und Konzeption über das Design und die eigentliche Implementierung bis hin zu intensiven Testläufen. Insgesamt beträgt der Arbeitsaufwand rund 25 bis 30 Stunden.

1.2 Kernfunktionen

Die wesentlichen Spielfunktionen lassen sich wie folgt zusammenfassen:

- Bei jedem Start wird ein Labyrinth zufällig erstellt
- Spieler, Schatz und Hindernisse werden automatisch im Spielfeld positioniert
- Der Spieler steuert seine Figur über die Konsole mithilfe der Tasten W, A, S und D
- Ungültige Züge & Eingaben werden erkannt und mit einer entsprechenden Meldung abgefangen
- Erreicht der Spieler den Schatz, endet das Spiel mit einer Siegesnachricht

1.3 Technische Umsetzung

Die Implementierung erfolgte konsequent modular. Der Quellcode wurde in separate Module für Labyrinth, Spieler und Spiellogik aufgeteilt, wodurch eine klare Struktur und einfache Wartbarkeit gewährleistet wird. Neben der Umsetzung der Kernfunktionen lag der Fokus auf robuster Fehlerbehandlung, sauberer Speicherverwaltung und der Vermeidung von Lecks beim wiederholten Spielstart. Durch diesen Ansatz konnte ein stabiler und erweiterbarer Programmaufbau erreicht werden.

1.4 Testphase

Das Programm wurde systematisch aufgebaut und nach jedem Zwischenschritt überprüft. Durch kontinuierliches Testen während der Entwicklung konnte die Funktionalität schrittweise sichergestellt werden. Anschliessend wurden vollständige Spielabläufe simuliert und getestet. Alle dabei entdeckten Probleme wurden behoben, so, dass ein stabiles und zuverlässiges Programm entstand.

1.5 Projektergebnisse

Das Projekt wurde erfolgreich abgeschlossen und alle in der Anforderungsanalyse definierten Kriterien wurden erfüllt. Das resultierende Spiel ist funktional, benutzerfreundlich und bietet eine solide technische Basis für Erweiterungen. Darüber hinaus wurden neben den geforderten Mindestanforderungen weitere Funktionen umgesetzt. Dazu zählen eine optimierte Speicherverwaltung zur Vermeidung von Lecks, eine erweiterte Fehlerbehandlung mit klaren Rückmeldungen sowie die Möglichkeit für den Spieler, die Größe des Labyrinths und dessen Schwierigkeitsgrad festzulegen. Ergänzend wurde auch eine Abbruchfunktion integriert, die es erlaubt, das Spiel jederzeit geordnet zu beenden.

1.6 Lessons Learned

Wichtige Erkenntnisse aus dem Projekt sind:

- Modularer Code erleichtert Wartung, Erweiterung und Fehlersuche erheblich
- Umfassende Tests und gründliche Fehlerbehandlung sind zentrale Erfolgsfaktoren
- Inkrementelle Entwicklung mit kurzen Feedbackschleifen verbessert Qualität und Fortschritt
- Kontinuierliche Dokumentation unterstützt Debugging und Nachvollziehbarkeit

1.7 Fazit

Insgesamt kann das Projekt als voller Erfolg gewertet werden. Es ermöglichte die Umsetzung theoretischer Inhalte in ein funktionierendes Produkt und schuf eine solide Basis für künftige Weiterentwicklungen. Vorstellbar sind etwa verschiedene Schwierigkeitsstufen, neue Spielfiguren oder anspruchsvollere Labyrinth-Layouts. Damit wurden nicht nur die gesetzten Ziele erreicht, sondern durch zusätzliche Funktionen sogar übertroffen.

2 Aufgabenstellung und Mindestanforderungen

2.1 Aufgabenstellung

Im Rahmen dieser Praxisarbeit wird die Entwicklung eines textbasierten Labyrinthspiels als Konsolenanwendung in der Programmiersprache C umgesetzt. Der Schwerpunkt liegt dabei auf der praktischen Anwendung der in den Modulen Programmiertechnik A und Programmiertechnik B erlernten Inhalte. Das Projekt verfolgt das Ziel, die theoretischen Kenntnisse in den Bereichen Algorithmen, Datenstrukturen und Programmsteuerung in einem praxisnahen Beispiel zu vertiefen.

Das Kernprinzip des Spiels besteht darin, dass der Spieler in einem Labyrinth einen verborgenen Schatz aufspüren muss. Die Spielfläche wird als zweidimensionales Array abgebildet, das die Struktur des Labyrinths modelliert. Dabei übernehmen bestimmte Symbole feste Rollen: „P“ kennzeichnet den Spieler, „T“ steht für den Schatz und „O“ markiert Hindernisse. Da diese Elemente bei jedem Spielstart zufällig verteilt werden, ergibt sich stets eine neue Ausgangssituation, was für mehr Abwechslung und Dynamik im Spiel sorgt.

Für die erfolgreiche Umsetzung des Projekts galt es, verschiedene Aspekte miteinander zu verbinden. Neben der Entwicklung eines lauffähigen und benutzerfreundlichen Spiels, das sämtliche Mindestanforderungen erfüllt, stand auch die Qualität des Programmcodes im Vordergrund. Ein modularer Aufbau, saubere Dokumentation und Wartbarkeit bilden die Grundlage dafür, dass das Ergebnis nicht nur als einmalige Abgabe dient, sondern auch für künftige Erweiterungen genutzt werden kann.

Zusammengefasst besteht die Aufgabe darin, ein Labyrinthspiel zu entwickeln, in dem der Spieler über Tastatureingaben gesteuert wird. Dabei gilt es, Hindernissen auszuweichen, bis der Schatz erreicht ist. Das Programm soll durch eine klare Benutzerführung überzeugen, Eingabefehler zuverlässig abfangen und das Spiel mit einer eindeutigen Siegesmeldung beenden.

2.2 Funktionsanforderungen

Die Mindestanforderungen an das Spiel ergeben sich direkt aus der Aufgabenstellung und bilden die Grundlage für die Implementierung. Sie lassen sich in funktionale und qualitative Anforderungen unterteilen.

Funktionale Anforderungen:

Darstellung des Spielfelds: Das Labyrinth wird in Form eines zweidimensionalen Arrays erzeugt. Spieler, Schatz und Hindernisse werden mit den Zeichen „P“, „T“ und „O“ gekennzeichnet. Ihre Positionen sind zufällig, sodass jede Partie einen neuen Spielverlauf ermöglicht.

Bewegung des Spielers: Die Spielfigur wird durch die Eingaben W (oben), A (links), S (unten) und D (rechts) bewegt. Jede Eingabe muss mit der Return-Taste bestätigt werden. Nach jeder Aktion wird das gesamte Spielfeld auf der Konsole neu ausgegeben, wodurch der Spieler jederzeit seine Position und den aktuellen Stand des Spiels nachvollziehen kann.

Kollisionserkennung: Bewegungen des Spielers werden so geprüft, dass weder Hindernisse durchschritten noch die Spielfeldgrenzen überschritten werden können. Falsche Eingaben lösen eine Rückmeldung an den Spieler aus, anstatt das Programm zu beenden, wodurch ein konsistentes und verständliches Spielerlebnis gewährleistet wird.

Siegbedingung: Erreicht der Spieler das Schatzfeld, gibt das Programm eine eindeutige Erfolgsmeldung aus und beendet das Spiel regulär.

Qualitative Anforderungen: Neben den spielrelevanten Grundfunktionen sind auch allgemeine Umsetzungsrichtlinien einzuhalten.

Dazu gehören:

Benutzerfreundlichkeit: Das Spiel muss für Einsteiger verständlich sein. Klare Eingabeaufforderungen und leicht nachvollziehbare Statusmeldungen unterstützen den Spieler während des gesamten Ablaufs.

Robustheit: Das Programm muss auch bei ungültigen Eingaben oder unerwarteten Spielverläufen stabil bleiben. Eine zuverlässige Fehlerbehandlung ist hierfür zwingend erforderlich.

Modularität und Wartbarkeit: Der Quellcode wird in klar getrennte Module aufgeteilt (z. B. Spiellogik, Labyrinth-Generierung, Benutzereingaben). Dies erhöht die Übersichtlichkeit und erleichtert Erweiterungen.

Effizienz: Der Code soll so implementiert werden, dass Speicher sparsam genutzt wird und die Eingaben des Spielers unmittelbar verarbeitet werden können. Dies trägt zu einem flüssigen Spielablauf bei.

Erweiterbarkeit: Das Projekt ist so anzulegen, dass zukünftige Anpassungen wie unterschiedliche Labyrinthgrößen, alternative Layouts oder zusätzliche Spielelemente mit überschaubarem Aufwand realisiert werden können.

Mit diesen Anforderungen wird sichergestellt, dass das Spiel nicht nur die Minimalfunktionen erfüllt, sondern zugleich eine solide technische Basis bietet, um den Lerneffekt und die Praxistauglichkeit zu maximieren.

3 Design

3.1 Entwurfsprozess

Der Entwurfsprozess orientierte sich weitgehend an der Aufgabenstellung. Festgelegt waren u. a. eine Konsolenanwendung in C, die Abbildung des Spielfelds als zweidimensionales Array, die Symbolik („P“ für den Spieler, „T“ für den Schatz, „O“ für Hindernisse), zufällige Startpositionen sowie die Steuerung über W/A/S/D mit anschließender Neuzeichnung und die Prüfung der Siegesbedingung.

Der verbleibende Gestaltungsspielraum lag vor allem in der internen Struktur und Qualitätssicherung: klare Modultrennung (Labyrinth, Spieler, Spiellogik), definierte Schnittstellen, robuste Eingabeprüfung und Kollisionserkennung, effiziente Aktualisierung der Darstellung, sorgfältige dynamische Speicherverwaltung sowie eine Abbruchfunktion für ein geordnetes Beenden. So entstand trotz enger Vorgaben ein wartbares Design, das die Anforderungen zuverlässig und nachvollziehbar umsetzt.

Parallel dazu wurden Überlegungen zur Benutzerfreundlichkeit angestellt. Ziel war es, ein Steuerungssystem zu entwickeln, das einfach erlernbar ist und auch von Spielern ohne Vorkenntnisse problemlos bedient werden kann. Die Wahl der Tasten W, A, S und D orientiert sich an den Vorgaben und erleichtert die intuitive Bedienung.

Um den Ablauf des Spiels von Beginn an übersichtlich zu gestalten, wurde der Prozess mithilfe von Diagrammen visualisiert. Auf diese Weise konnten mögliche Probleme schon vor der eigentlichen Programmierung identifiziert und berücksichtigt werden. Das Aktivitätsdiagramm veranschaulicht die wichtigsten Entscheidungspfade des Spiels, darunter die Eingabeprüfung, die Erkennung von Hindernissen sowie die Abfrage der Siegesbedingung.

3.2 Gewählte Lösung

Die endgültige Lösung basiert auf einem zweidimensionalen Array, das als Datenstruktur für das Labyrinth dient. Jedes Feld dieses Arrays repräsentiert ein Element des Spielfelds. Durch die Verwendung einfacher Symbole („P“ für den Spieler, „T“ für den Schatz, „O“ für Hindernisse) wird eine klare und leicht verständliche Darstellung erreicht. Der Vorteil dieser Struktur liegt in der einfachen Verwaltung: Positionen können direkt über Indexwerte angesprochen werden, und Operationen wie Verschieben oder Prüfen von Nachbarfeldern lassen sich effizient implementieren.

Bei der Konzeption des Codes wurde ein modularer Aufbau verfolgt. Geplant war eine Trennung in verschiedene Funktionsbereiche:

- **Labyrinth-Generierung:** Erstellung und Initialisierung des Spielfelds.
- **Eingabeverarbeitung:** Steuerung des Spielers über Tastatureingaben.
- **Spiellogik:** Überprüfung von Bewegungen, Kollisionen und Siegesbedingungen.
- **Ausgabe:** Aktualisierung und Darstellung des Spielfelds in der Konsole.

Diese klare Struktur erleichtert sowohl das Verständnis als auch die spätere Wartung und Erweiterung des Programms.

```
// Funktion zum Initialisieren des Labyrinths
void initialisiereLabyrinth(Labyrinth *lab, int zeilen, int spalten)
{
    lab->zeilen = zeilen;           // Zeilenanzahl setzen
    lab->spalten = spalten;         // Spaltenanzahl setzen

    lab->feld = (char **)malloc(zeilen * sizeof(char *)); // Speicher für Zeilen reservieren
    for (int i = 0; i < zeilen; i++)
    {
        lab->feld[i] = (char *)malloc(spalten * sizeof(char)); // Speicher für Spalten reservieren
        for (int j = 0; j < spalten; j++)
        {
            lab->feld[i][j] = '.'; // Leeres Feld initialisieren
        }
    }

    lab->spielerPosX = lab->spielerPosY = -1; // Spielerposition initialisieren
    lab->schatzPosX = lab->schatzPosY = -1;  // Schatzposition initialisieren
}
```

Bild: Initialisierung des Labyrinths

3.3 Detailbeschreibung der Lösung

Die Spiellogik des Programms basiert auf wiederholten Eingaben des Spielers. Nach jeder Eingabe wird überprüft, ob die Bewegung gültig ist. Ungültige Züge wie etwa der Versuch, durch eine Wand zu laufen oder das Spielfeld zu verlassen, werden vom Programm blockiert und mit einer Meldung quittiert. Dadurch wird verhindert, dass der Spieler durch fehlerhafte Eingaben den Spielablauf stört oder das Programm abstürzt.

Nach jeder gültigen Eingabe wird die Spielerposition angepasst und das Spielfeld in der Konsole erneut ausgegeben. Dadurch lassen sich die Bewegungen unmittelbar nachvollziehen. Dieser Ablauf aus Eingabe, Validierung, Aktualisierung und Darstellung wiederholt sich fortlaufend, bis entweder der Schatz erreicht oder das Spiel vorzeitig beendet wird.

Besondere Aufmerksamkeit wurde der Platzierung der Hindernisse gewidmet. Tests haben gezeigt, dass eine zufällige Anordnung der Hindernisse für ein abwechslungsreicheres Spielerlebnis sorgt, da der Spieler bei jedem neuen Spiel mit einer unbekannten Situation konfrontiert wird.

Die Siegesbedingung wurde in einer klaren und unkomplizierten Form umgesetzt. Das Spiel endet automatisch, sobald die Spielfigur das Feld mit dem Schatz erreicht. In diesem Moment wird eine eindeutige Siegesmeldung ausgegeben, die den erfolgreichen Abschluss der Partie signalisiert und den Spieler darüber informiert, dass sein Ziel erreicht wurde. Damit ist der Spielablauf eindeutig geregelt und das Ende jederzeit transparent nachvollziehbar.

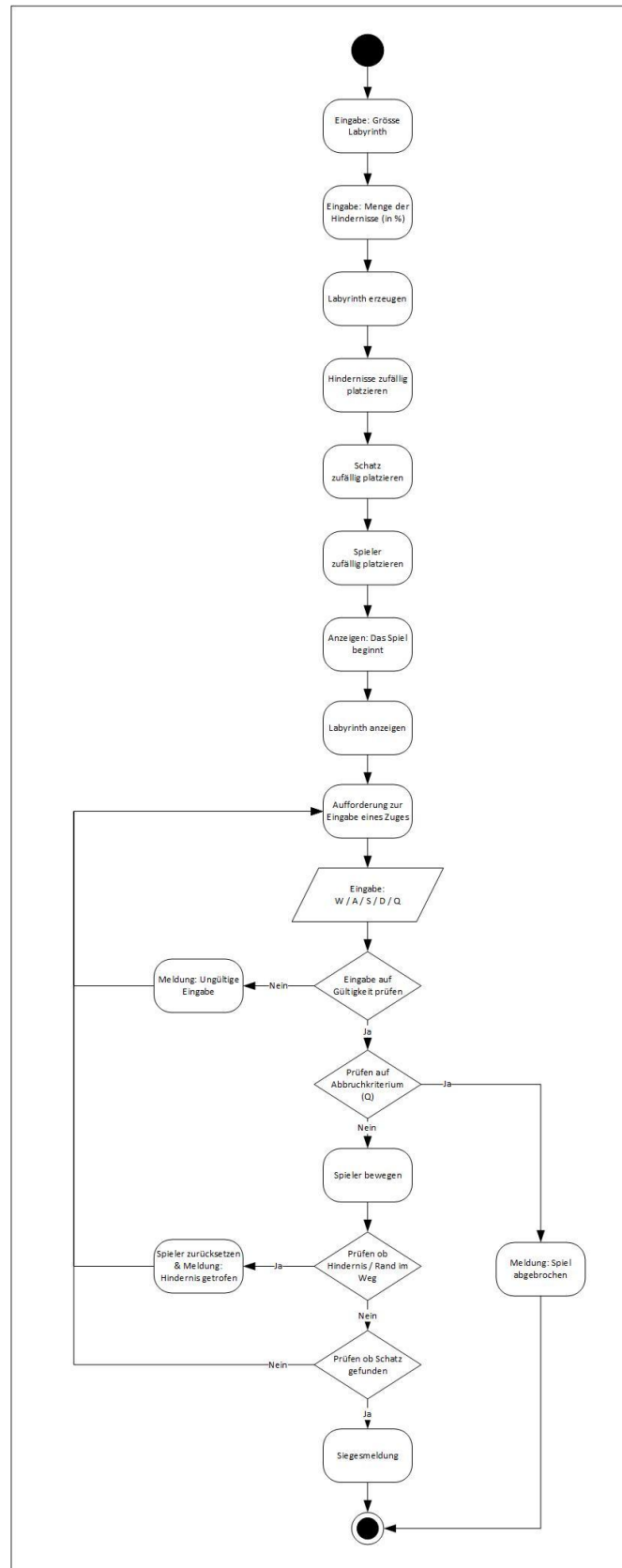


Bild: Aktivitätsdiagramm fertiges Programm

4 Implementierung

4.1 Projektstruktur

Die Implementierung wurde so angelegt, dass das Projekt in mehrere klar abgegrenzten Module unterteilt ist. Diese Modularisierung stellt sicher, dass der Quellcode übersichtlich bleibt und einzelne Teile unabhängig voneinander entwickelt, getestet und bei Bedarf angepasst werden können.

Die Struktur gliedert sich in folgende Hauptbestandteile:

- **main.c:** Einstiegspunkt des Programms. Hier erfolgt die Eingabe und Validierung der Startparameter (z. B. Spielfeldgröße, Hindernisdichte), die Initialisierung des Spiels sowie die Steuerung der Hauptschleife.
- **labyrinth.c / labyrinth.h:** Funktionen zur Generierung und Verwaltung des Spielfelds, Platzierung von Spieler, Schatz und Hindernissen sowie Ausgabe des Labyrinths in der Konsole.
- **spieler.c / spieler.h:** Realisierung der Spielerbewegungen inklusive Kollisionserkennung und Prüfung der Siegbedingung.

Durch die Aufteilung in Header-Dateien (.h) und Implementierungsdateien (.c) wurde eine saubere Trennung zwischen Schnittstellen und Funktionalitäten erreicht. Diese Struktur erleichtert nicht nur die Lesbarkeit, sondern unterstützt auch eine zukünftige Erweiterung.

4.2 Kernfunktionen

Die wichtigsten Funktionen des Spiels lassen sich in drei Hauptkategorien unterteilen: **Initialisierung**, **Eingabeverarbeitung** und **Spielablauf**.

- **Initialisierung:** Zu Beginn des Spiels wird das Spielfeld erzeugt. Der Spieler und der Schatz werden an zufälligen Positionen gesetzt. Hindernisse werden ebenfalls zufällig verteilt, wobei darauf geachtet wird, dass Schatz und Spieler nicht blockiert sind.
- **Eingabeverarbeitung:** Die Benutzereingabe erfolgt über die Tasten W, A, S und D. Nach jeder Eingabe prüft das Programm, ob die Bewegung gültig ist. Falls ja, wird die Spielfigur an die neue Position verschoben. Andernfalls bleibt der Spieler stehen und erhält eine Fehlermeldung.
- **Spielablauf:** Nach jeder Eingabe wird das Spielfeld neu ausgegeben. Der Zyklus aus Eingabe, Überprüfung und Darstellung wiederholt sich, bis die Siegbedingung erfüllt ist.

Ein Beispiel für eine zentrale Funktion ist die **Kollisionserkennung**: Hier wird überprüft, ob die Zielposition innerhalb der Grenzen des Arrays liegt und nicht durch ein Hindernis blockiert ist.

4.3 Optimierung

Nachdem die Grundfunktionen stabil liefen, wurde der Fokus auf Effizienz und Benutzerfreundlichkeit gelegt. Ziel war es, kurze Reaktionszeiten zu gewährleisten und die Interaktion möglichst flüssig zu gestalten.

Zu den wichtigsten Optimierungen zählen:

- **Effiziente Spielfeldaktualisierung:** Statt das gesamte Array bei jeder Eingabe komplett neu zu berechnen, werden nur die relevanten Positionen angepasst.
- **Speicherverwaltung:** Durch gezielte Verwendung von dynamischer Speicherallokation wird unnötiger Ressourcenverbrauch vermieden.
- **Konsolenausgabe:** Die Darstellung des Spielfelds wurde so optimiert, dass das Labyrinth übersichtlich bleibt, ohne die Konsole zu überfüllen.

Diese Anpassungen tragen dazu bei, dass das Spiel auch bei wiederholtem Neustart stabil bleibt und den Spieler nicht durch unnötige Verzögerungen aus dem Spielfluss reisst.

```

Legende:
P = Spieler
T = Schatz
O = Hindernis
. = frei

. . . . . O . . O . . . . . O O O . .
. . O . . O . . . . . . . . . . . .
. . O . O . . . . . O O . . . . . O .
. . . . . . . . O . O . . . . . O . O .
O . . . . O . . O O . O . . . . . O . .
. . O . . . . . O . . . . . O . . . . O
. . . . . . O . O . . . . . T . . . . .
. . . O O . . O . . O . . O . O O . .
O . . . . . . . O . . . . . . . . . O .
. . O . . . . O . O . O . . . . . O . .
. . . . O . . . . . . . O . O . . . .
O O . . O . . O . O . O . . . . . O .
. . . O . O O . O O . O O . O . . . .
. . . . O . . . . . . . O . O . . . .
. . . O O O . O . . . . . O . . . O O
. . O . . O . . O O . . . . . O . O .
. . . O . . . . . O . . . . . O . .
O O . . . . O . . . . O . . . . . O .
O . . . . O . . . . . O O . . . . .
. O . . . . . . . . . O . O O . O .

Steuerung:
W = Hoch
A = Links
S = Runter
D = Rechts
Q = Spiel abbrechen

Gib die Richtung deines nächsten Zugs ein:

```

Bild: Ausgabe des Labyrinths auf dem Terminal

4.4 Erweiterungen

Obwohl das Spiel die Mindestanforderungen bereits erfüllt, wurden zusätzliche Features integriert, die den Funktionsumfang über die Vorgabe hinaus erweitern:

- **Einstellungen Labyrinth:** Vor Spielbeginn kann der Spieler sowohl die Größe des Spielfelds als auch den prozentualen Anteil an Hindernissen festlegen. Auf diese Weise lässt sich der Schwierigkeitsgrad variieren und das Spiel an die eigenen Vorlieben anpassen.
- **Anzahl Spielzüge:** Am Ende des Spiels wird die Anzahl der benötigten Züge angezeigt, was zusätzliche Motivation schafft und einen Vergleich zwischen verschiedenen Durchläufen erlaubt.
- **Erweiterte Fehlerbehandlung:** Neben der Abfrage gültiger Eingaben wurden informative Rückmeldungen integriert, die den Spieler über falsche Eingaben aufklären.

Darüber hinaus ist der Code so gestaltet, dass künftige Erweiterungen wie z.Bsp. mehrere Schatzobjekte, oder zusätzliche Spielfiguren, mit überschaubarem Aufwand ergänzt werden können.

```
srand((unsigned int)time(NULL)); // Initialisiere Zufallsgenerator

int zeilen = 10, spalten = 10, hindernisProzent = 20; // Standardwerte

printf("\n\n----- Labyrinth-Spiel ----- \n\n"); // Titel des Spiels
printf("Gib die Grösse des Feldes an (z.Bsp. 12 12). Leer lassen für Standardgrösse (10 x 10). \nWunschgrösse: "); // Eingabeaufforderung

int gelesen = 0; // Variable, um zu überprüfen, ob eine gültige Grösse eingegeben wurde
char buf[64]; // Puffer für die Eingabe

if (fgets(buf, sizeof(buf), stdin)) // Lese die Eingabe
{
    int z, s; // Temporäre Variablen für Zeilen und Spalten
    if (sscanf(buf, "%d %d", &z, &s) == 2 && z >= 5 && z <= 99 && s >= 5 && s <= 99) // Überprüfe, ob die Eingabe gültig ist
    {
        zeilen = z; spalten = s; gelesen = 1; // Setze die Grösse und markiere als gelesen
    }
}

if (gelesen) // Wenn eine gültige Grösse eingegeben wurde
{
    printf("Eingelesene grösse des Labyrinths: %d x %d \n", zeilen, spalten);
}

if (!gelesen) // Wenn keine gültige Grösse eingegeben wurde
{
    printf("Es wird die Standardgrösse %d x %d verwendet\n", zeilen, spalten);
}
```

Bild: Code-Erweiterung zur manuellen Eingabe der Labyrinthgösse



Bild: Ausgabe bei gewonnen Spiel mit Anzeige der benötigten Züge

5 Test

5.1 Modultests

Zu Beginn wurden einzelne Programmteile isoliert getestet, um deren Funktionalität unabhängig vom restlichen System sicherzustellen.

- Beim Modul *Labyrinthgenerierung* wurde geprüft, ob Spieler („P“), Schatz („T“) und Hindernisse („O“) korrekt zufällig platziert werden und keine Überschneidungen entstehen.
- Das Modul *Eingabeverarbeitung* wurde gezielt mit gültigen (W, A, S, D, Q) und ungültigen Eingaben getestet. Hierbei zeigte sich, dass ungültige Eingaben korrekt abgefangen und mit einer Meldung quittiert werden.
- Bei der *Kollisionserkennung* wurden gezielt Szenarien erstellt, in denen der Spieler auf Hindernisse oder die Spielfeldgrenze zuläuft. Es wurde überprüft, ob die Bewegung blockiert und der Spieler an seiner ursprünglichen Position bleibt.

Diese Modultests konnten mit einfachen Testläufen über die Konsole durchgeführt werden. Durch die gezielte Eingabe bestimmter Bewegungsfolgen liessen sich die Reaktionen des Programms eindeutig nachvollziehen.

5.2 Systemtests

Nach erfolgreichem Abschluss der Modultests folgte die Überprüfung des Gesamtsystems. Dabei wurde der vollständige Spielablauf simuliert, vom Start über mehrere Bewegungen bis hin zum Finden des Schatzes.

Besondere Aufmerksamkeit galt folgenden Aspekten:

- **Korrekte Spielfeldaktualisierung:** Nach jedem Zug musste das Spielfeld konsistent neu ausgegeben werden, sodass die neue Position des Spielers klar ersichtlich war.
- **Verhalten bei Grenzfällen:** Hierbei wurden Szenarien getestet, in denen der Spieler mehrfach hintereinander gegen eine Wand oder ein Hindernis läuft. Das Programm musste dies korrekt erkennen, den Spieler blockieren und gleichzeitig eine verständliche Rückmeldung geben.
- **Siegbedingung:** Das Auffinden des Schatzes wurde in unterschiedlichen Spielsituationen getestet, sowohl in freier Umgebung als auch in Kombination mit angrenzenden Hindernissen. In allen Fällen reagierte das Programm korrekt, gab die vorgesehene Siegesmeldung aus und beendete das Spiel wie vorgesehen.

Die Systemtests zeigen, dass die verschiedenen Module sauber ineinandergreifen und das Programm in seiner Gesamtheit die geforderten Funktionen erfüllt.

```
Steuerung:
W = Hoch
A = Links
S = Runter
D = Rechts
Q = Spiel abbrechen

Gib die Richtung deines nächsten Zugs ein: a
Du bist auf ein Hindernis gestossen
-----
```

Bild: Fehlermeldung bei touchieren eines Hindernisses

```
Steuerung:  
W = Hoch  
A = Links  
S = Runter  
D = Rechts  
Q = Spiel abbrechen  
  
Gib die Richtung deines nächsten Zugs ein: dd  
Bewegung ausserhalb des Labyrinths!  
-----
```

Bild: Fehlermeldung bei touchieren des Labyrinth-Rand

5.3 Zusammenfassung

Insgesamt haben die Tests gezeigt, dass das Labyrinthspiel die geforderten Mindestanforderungen erfüllt und in zentralen Bereichen robust und zuverlässig arbeitet. Alle identifizierten Fehler, insbesondere bei der Kollisionserkennung und Eingabeverarbeitung, wurden behoben. Das Endprodukt überzeugt durch Stabilität, Benutzerfreundlichkeit und einen nachvollziehbaren Spielablauf.

6 Lessons Learned

Wichtig Aspekte sind:

- Modularer Aufbau erleichtert Wartung und Erweiterbarkeit
- Umfassende Tests sichern die Stabilität
- Eine schrittweise Entwicklung spart Zeit und verbessert die Qualität
- Kontinuierliche Dokumentation unterstützt die Nachvollziehbarkeit

Im Rückblick lässt sich festhalten, dass der schrittweise Entwicklungsprozess entscheidend für den Erfolg des Projekts war. Durch die fortlaufende Überprüfung einzelner Module konnten Fehler frühzeitig identifiziert und behoben werden. Ebenso erwies sich eine strukturierte Dokumentation als wesentlich, da sie sowohl die Nachvollziehbarkeit sicherstellte als auch eine effektive Unterstützung bei der Fehlersuche bot. Besonders die modulare Architektur erwies sich als Vorteil, da sie eine solide Basis für Erweiterungen schafft. Ein zentrales Ergebnis ist zudem die Erkenntnis, dass eine sorgfältige Planung und Konzeption zu Beginn entscheidend ist, um Verzögerungen zu vermeiden. Die Arbeit mit der Programmiersprache C verdeutlichte ausserdem, wie wichtig ein präziser Umgang mit Speicherverwaltung ist. Darüber hinaus zeigte sich, dass die Verbindung von Theorie und Praxis die Lernkurve deutlich steigert und das Verständnis komplexer Inhalte vertieft. Ergänzend dazu wurde klar, dass eine präzise Aufgabenstellung, eine realistische Zeitplanung sowie die Aufteilung in Meilensteine entscheidend zum Fortschritt beitragen.

7 Anhang

- C-Projekt (Git-Hub Repository)
- Aktivitätsdiagramm
- Arbeitsjournal